

Documentación Técnica: Proyecto de Ingeniería de Datos E-commerce

Mark Brayam Renteria Morales

4 de mayo de 2026

1. Introducción

Este proyecto presenta una solución integral de analítica moderna (Modern Data Stack) para un ecosistema de e-commerce. El objetivo es transformar datos crudos con inconsistencias en activos de datos confiables para la toma de decisiones, utilizando **dbt** para la lógica de transformación, **BigQuery** como motor de procesamiento y **Apache Airflow** como el cerebro de la orquestación.

2. Arquitectura del Proyecto

Se implementó una arquitectura de capas diseñada para escalar y facilitar el mantenimiento de la lógica de negocio:

- **Capa de Ingesta (Raw):** Datos originales almacenados en Google BigQuery.
- **Capa de Transformación (dbt):** Procesamiento modular dividido en Staging, Intermediate y Marts.
- **Capa de Orquestación (Airflow):** Control del flujo de datos mediante un Grafo Acíclico Dirigido (DAG) que garantiza la ejecución secuencial y el manejo de errores.

3. Sección 1 - Modelado y Transformación con dbt

La estructura de carpetas sigue el principio de separación de responsabilidades:

- **Staging:** Es el punto de entrada. Aquí realizamos el "casteo" de tipos de datos (por ejemplo, convertir texto a fechas) y renombramos columnas para mantener consistencia en todo el proyecto.
- **Intermediate:** En esta capa realizamos los *Joins* complejos. Aquí es donde se une la información de pedidos con los detalles de productos para calcular métricas base sin exponerlas aún al usuario final.

- **Marts:** Son las tablas de salida optimizadas para herramientas de BI (como Looker o Tableau).
 - `mart_orders_summary`: Una visión consolidada de las ventas.
 - `mart_customer_ltv`: Un modelo que calcula el valor de vida del cliente (Lifetime Value) y los segmenta usando lógica de negocio.

4. Sección 2 - Calidad de Datos (Data Contracts)

Para garantizar que los reportes sean precisos, el pipeline actúa como un filtro de calidad:

- **Tests Genéricos:** Aplicados directamente en los archivos YAML. Validamos que no existan duplicados en las llaves primarias (*unique*), que no haya campos vacíos en datos críticos (*not_null*) y que cada venta pertenezca a un cliente real (*relationships*).
- **Tests Singulares (Lógica de Negocio):** Implementamos validaciones personalizadas mediante SQL. Por ejemplo, el archivo `assert_revenue_is_positive.sql` asegura que no se procesen ventas con montos negativos, lo cual indicaría un error en el sistema de origen.
- **Source Freshness:** dbt verifica automáticamente si los datos en BigQuery se han actualizado en las últimas 24 horas. Si los datos están “estancados”, el proceso se detiene antes de generar reportes obsoletos.

5. Sección 3 - Orquestación con Apache Airflow

El flujo de datos se automatiza mediante un DAG con una política de “**Test-Early**” (Probar temprano). La lógica del flujo es:

1. Verificar frescura de los datos originales.
2. Procesar y validar la capa de Staging.
3. Si las validaciones pasan, procesar los Marts finales.
4. Ejecutar pruebas de lógica de negocio y generar la documentación del linaje.

6. Análisis SQL: Rendimiento de Productos

La siguiente consulta identifica el rendimiento mensual de los productos, permitiendo detectar tendencias de crecimiento:

```

1 WITH monthly_sales AS (
2     -- Agregamos ventas por mes y producto
3     SELECT
4         FORMAT_DATE('%Y-%m', o.order_date) as mes,
```

```

5         p.name as producto,
6         SUM(oi.quantity * p.price) as revenue
7     FROM {{ ref('stg_order_items') }} oi
8     JOIN {{ ref('stg_products') }} p ON oi.product_id = p.product_id
9     JOIN {{ ref('stg_orders') }} o ON oi.order_id = o.order_id
10    GROUP BY 1, 2
11 )
12 -- Aqui se podria aplicar una ventana (Window Function) para el Top
    5

```

7. Entregables

1. Repositorio

Se genero un repositorio para el codigo con los archivos necesarios, se añaden archivos criticos de configuración aparte en un rar por correo.

<https://github.com/markbrayam/e-commerce.git>

2. Archivo DAG

```

1     from airflow import DAG
2     from airflow.operators.bash import BashOperator
3     from datetime import datetime, timedelta
4
5     # Rutas validadas en el contenedor
6     DBT_PROJECT_DIR = "/opt/airflow/dbt-project"
7     DBT_PROFILES_DIR = "/opt/airflow"
8
9     default_args = {
10         'owner': 'data_engineering',
11         'depends_on_past': False,
12         'email_on_failure': False,
13         'email_on_retry': False,
14         'retries': 1,
15         'retry_delay': timedelta(minutes=5),
16     }
17
18     with DAG(
19         'dag_ecommerce_dbt_transformation',
20         default_args=default_args,
21         description='Pipeline optimizado: Freshness >
22             Staging > Test > Marts > Test',
23         schedule_interval='0 2 * * *',
24         start_date=datetime(2026, 4, 1),
25         catchup=False,
26         tags=['bi', 'dbt', 'ecommerce', 'production'],
27     ) as dag:

```

```

27 # 1. VERIFICACION DE FRESCURA (Gatekeeper 1)
28 # Si los datos en BigQuery son m s viejos de lo
29     definido en tu source.yml, el DAG se detiene.
30 check_freshness = BashOperator(
31     task_id='dbt_source_freshness',
32     bash_command=f'cd {DBT_PROJECT_DIR} && dbt
        source freshness --profiles-dir {
        DBT_PROFILES_DIR}'
33 )
34
35 # 2. RUN & TEST STAGING (Gatekeeper 2)
36 # Limpiamos los datos crudos y validamos
        inmediatamente NULOS y UNICIDAD.
37 # Si el test falla, no se gasta procesamiento en
        modelos pesados de Marts.
38 run_test_staging = BashOperator(
39     task_id='dbt_run_test_staging',
40     bash_command=(
41         f'cd {DBT_PROJECT_DIR} && '
42         f'dbt run --select staging --profiles-dir {
            DBT_PROFILES_DIR} && '
43         f'dbt test --select staging --profiles-dir
            {DBT_PROFILES_DIR}'
44     )
45 )
46
47 # 3. RUN TRANSFORMATION (Core Logic)
48 # Ejecuta modelos Intermediate y Marts. dbt maneja
        las dependencias internas.
49 run_marts = BashOperator(
50     task_id='dbt_run_marts',
51     bash_command=f'cd {DBT_PROJECT_DIR} && dbt run
        --select intermediate marts --profiles-dir {
        DBT_PROFILES_DIR}'
52 )
53
54 # 4. TEST BUSINESS LOGIC (Final Guard)
55 # Ejecuta tus tests singulares (como
        assert_revenue_is_positive.sql) y validaciones
        de Marts.
56 test_marts = BashOperator(
57     task_id='dbt_test_business_logic',
58     bash_command=f'cd {DBT_PROJECT_DIR} && dbt test
        --select marts --profiles-dir {
        DBT_PROFILES_DIR}'
59 )

```

```

60
61 # 5. DOCUMENTACION (Metadata)
62 # Genera el cat logo y el manifest.json final para
    el Lineage.
63 generate_docs = BashOperator(
64     task_id='dbt_generate_docs',
65     bash_command=f'cd {DBT_PROJECT_DIR} && dbt docs
        generate --profiles-dir {DBT_PROFILES_DIR}'
66 )
67
68 # Orden de ejecuci n l gica
69 check_freshness >> run_test_staging >> run_marts >>
    test_marts >> generate_docs

```

3. Captura del lineage graph de dbt

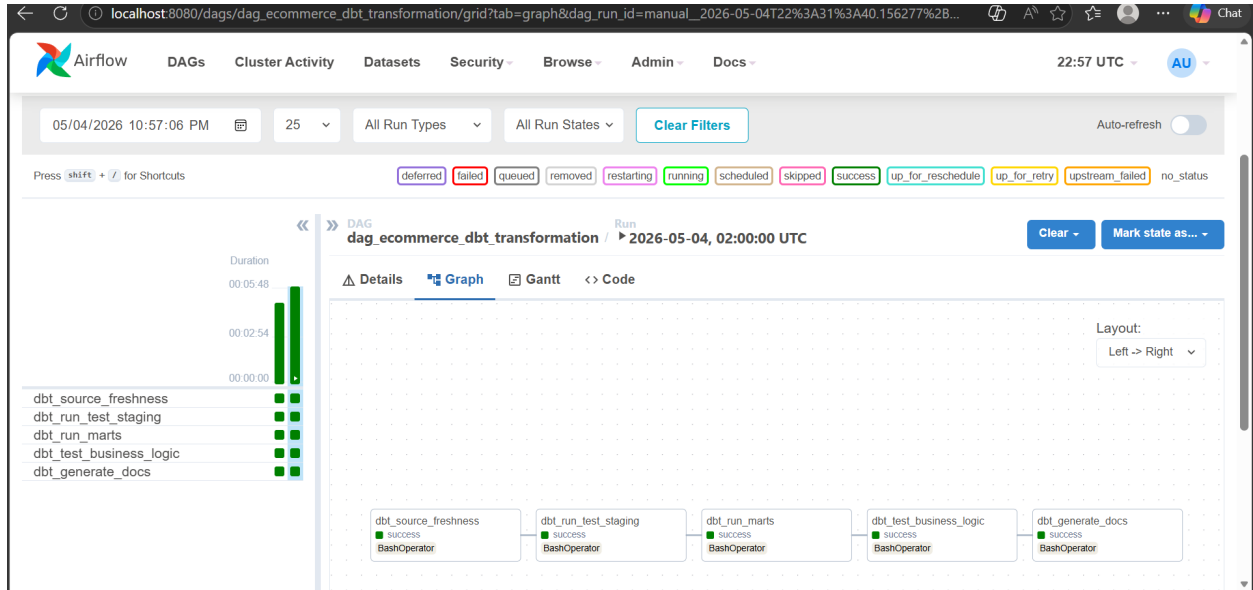


Figura 1: Lineage Graph



Figura 2: Airflow API

4. Query SQL del punto 4.2

```

1 WITH monthly_sales AS (
2     -- Agregamos ventas por mes y producto
3     SELECT
4         FORMAT_DATE('%Y-%m', o.order_date) as mes,
5         p.name as producto,
6         SUM(oi.quantity * p.price) as revenue
7     FROM {{ ref('stg_order_items') }} oi
8     JOIN {{ ref('stg_products') }} p ON oi.product_id = p.
9         product_id
10    JOIN {{ ref('stg_orders') }} o ON oi.order_id = o.order_id
11    GROUP BY 1, 2
12 )
13 -- Aqui se podria aplicar una ventana (Window Function) para el
14    Top 5

```

5. Readme.md

Se incluye readme.md en el repositorio de github

Se requiere tener instalado github y ejecutar el siguiente comando.

git clone https://github.com/markbrayam/e-commerce.git

8. Conclusión

La solución actual sienta las bases de un sistema robusto, pero la transición hacia una plataforma de datos de clase mundial requiere la implementación de pasos detallados en cuatro ejes fundamentales:

- **Automatización de la Calidad (Data Guardrails):** Implementar entornos de CI/CD donde cada Pull Request ejecute un `dbt test` integral antes de permitir la unión al código de producción.
- **Gobierno de Datos Activo:** Utilizar el linaje generado por dbt para alimentar catálogos de datos corporativos, permitiendo que los usuarios finales entiendan el origen (provenance) de cada KPI.
- **Optimización de Cómputo:** Establecer políticas de *Incremental Models* en dbt para procesar únicamente los deltas de datos diarios, reduciendo significativamente el tiempo de ejecución y el costo operativo en la nube.
- **Seguridad y Cumplimiento:** Integrar la gestión de secretos (como Google Secret Manager) en Airflow para manejar las credenciales de conexión sin exponer información sensible en el código fuente.
- La madurez del ecosistema permite la transición de una contabilidad descriptiva a una analítica financiera prescriptiva. Se propone la creación de un *Dashboard de Unit Economics* que consolide los siguientes pilares:
 1. **Conciliación de Márgenes:** Cálculo automatizado del *Gross Merchandise Value* (GMV) neto de impuestos y cancelaciones, asegurando que la dirección financiera trabaje con datos auditados por los tests de dbt.
 2. **Análisis de Cohortes (Retention Finance):** Evaluación del comportamiento de recompra por mes de adquisición para proyectar el flujo de caja futuro basado en la retención histórica.
 3. **Atribución de Gastos Operativos:** Prorratio de costos de logística y pasarelas de pago a nivel de producto individual, permitiendo identificar qué categorías del e-commerce son realmente rentables tras considerar costos ocultos.

Este nivel de detalle financiero es inalcanzable sin la orquestación de Airflow, que garantiza la puntualidad de los datos, y la lógica de negocio centralizada en dbt, que elimina las discrepancias entre departamentos.